Python O'rganish Qo'llanmasi

Barcha mavzular + qo'shimcha tushuntirishlar

Mukhammad | 2026

Mavzular ro'yxati:

1. Python nima? - Asosiy tushuncha

Python - yuqori darajali, o'qilishi oson, ko'p maqsadli dasturlash tili. 1991-yilda Guido van Rossum tomonidan yaratilgan. Python hozirda AI, web, automation va ilmiy sohalarda keng qo'llaniladi.

Python qayerlarda ishlatiladi?

- Web - Full Stack (Django & FastAPI)
- AI / Machine Learning (TensorFlow, PyTorch)
- Automation / Testing (Selenium)
- Desktop (Kivy, BeeWare)
- Kiberxavfsizlik (CSRF, Injection tahlili)

Python va Java/JS farqi

- Java/JS da o'zgaruvchi - xotira joyi nomi (storage location name)
- Python da o'zgaruvchi - ob'ektga yo'llanma (named reference)
- Pythonda hamma narsa ob'ekt: int, str, bool, list, modul...
- Python da blok uchun {} emas, indentation (bo'sh joy) ishlatiladi

```
# Pythonda hamma narsa ob'ekt!
import math
print(type('Hello'))   # <class 'str'>
print(type(100))       # <class 'int'>
print(type(True))      # <class 'bool'>
print(type(math))      # <class 'module'>

# Built-in qurollar:
# TYPES: int float str list dict tuple
# FUNCTIONS: print() len() input() type()
# CONSTANTS: True False None
print(dir(__builtins__))  # barcha built-in qurollarni ko'rish
```

Dasturlash paradigmalari

Python ikki asosiy paradigmani qo'llab-quvvatlaydi:

- Functional Programming - funksiyalar asosida
- OOP (Object Oriented Programming) - ob'ektlar asosida

OOP 4 ta asosiy konsepti: Abstraction | Encapsulation | Inheritance | Polymorphism

Eslatma: Python da __builtins__ - barcha tayyor qurollar joylashgan maxsus modul. print(dir(__builtins__)) orqali ularni ko'rishingiz mumkin.

2. Primitive turlar - int, str, bool

int (son)

Pythonda butun sonlar int tipida saqlanadi. Java/JS dan farqli o'laroq, Python da int ham ob'ekt hisoblanadi va uning o'z metodlari bor.

```
count = 100
print(type(count))           # <class 'int'>
print(count.bit_length())    # 7  (ikkilik sanoqda necha bit)
print(count.numerator)       # 100 (kasrning surati)
```

str (matn)

String - matn tipi. Qo'shtirnoq (' yoki ") ichida yoziladi.

```
course = 'AI Python FullStack'
print(type(course))                      # <class 'str'>
print(course.title())                    # Ai Python Fullstack
print(course.upper())                    # AI PYTHON FULLSTACK
print(course.lower())                    # ai python fullstack
print(course.replace('FullStack', 'ML')) # AI Python ML

# f-string (format string) - eng qulay usul
name = 'Ali'
age = 22
print(f'Mening ismim {name}, yoshim {age}')  # Mening ismim Ali, yoshim 22
```

bool (mantiqiy)

bool tipi faqat True yoki False qiymatlarni oladi. Pythonda har qanday qiymat TRUTHY yoki FALSY bo'lishi mumkin.

```
# TRUTHY qiymatlar: True, 100, -100, 'MIT' (nol bo'lmagan son, bo'sh bo'lmagan str)
# FALSY qiymatlar: False, 0, '', None

test_falsy = '' or False or None or 0
print(bool(test_falsy))     # False

test_truthy = 'MIT'
print(bool(test_truthy))    # True

# input() dan kelgan qiymat DOIM string!
y = input('Qiymat kiriting: ')
print(y.isnumeric())  # True/False
```

Maslahat: bool() funksiyasi yordamida istalgan qiymatni True/False ga aylantirish mumkin. Bu ayniqsa shartlarda va input validatsiyasida foydali.

None - bo'sh qiymat

```
# None - Python da 'hech narsa yo'q' degan ma'no
# Java dagi null, JS dagi undefined ga o'xshaydi
```

```
result = None
print(result)          # None
print(type(result))    # <class 'NoneType'>
print(bool(result))    # False (FALSY)
```

3. Operatorlar va Shartlar

Arifmetik operatorlar

```
a = 19
b = 5

print(a + b)    # 24  - qo'shish
print(a - b)    # 14  - ayirish
print(a * b)    # 95  - ko'paytirish
print(a / b)    # 3.8 - bo'lish (float)
print(a // b)   # 3   - butun bo'lish (floor division)
print(a % b)    # 4   - qoldiq (modulo)
print(b ** 2)   # 25  - daraja (power)
print(b ** 3)   # 125

# O'zlashtirish operatorlari
a += 100   # a = a + 100
a -= 10    # a = a - 10
a *= 2     # a = a * 2
```

Taqqoslash: == va is farqi

== - qiymatlarni solishtiradi (JavaScript dan farqli - JS da reference solishtiradi). is - bir xil ob'ektmi deb tekshiradi (reference).

```
c = dict(name='Adam', age=20)
d = dict(name='Adam', age=20)
e = c

print(c == d)   # True  (qiymatlar teng)
print(c is d)   # False (turli ob'ektlar)
print(e is c)   # True  (bir xil ob'ekt, e = c reference oldi)

print(id(c))    # 140234...  (ob'ektning xotira manzili)
print(id(d))    # 140235...  (boshqa manzil)
```

Eslatma: Python da == qiymatlarni solishtiradi (NodeJS dan farqli). is esa xotira manzilini solishtiradi. None tekshirishda 'x is None' ishlatish tavsiya etiladi.

Shartlar - if / elif / else

```
x = 5

if x > 50:
    print('case A')
elif x > 10:
    print('case B')
else:
    print('case C')    # bu chiqadi
```

```
# TERNARY (qisqa if-else)
age = 18
person = 'adult' if age > 18 else 'minor'
print(person)   # minor
```

Mantiqiy operatorlar - and, or, not

```
is_student = True
is_parent = False
is_guest = True
is_admin = False

if not is_student:
    print('Talaba emas')
elif is_admin:
    print('Admin')
elif is_guest or is_parent:
    print('Kutish xonasiga boring!')   # bu chiqadi
else:
    print('Boshqa holat')
```

Maslahat: not - mantiqni teskariga o'zgartiradi. or - birontasi True bo'lsa yetarli. and - ikkalasi ham True bo'lishi kerak.

4. Tsikllar - for va while

for tsikli

for tsikli iterable ob'ektlar ustidan aylanadi: string, list, tuple, dict, range, map, filter va boshqalar.

```
# String ustidan
for letter in 'MIT':
    print(letter)   # M, I, T

# List ustidan
numbs = [10, 5, 3, 1]
for number in numbs:
    print(number)

# range() bilan
for x in range(5):          # [0, 1, 2, 3, 4]
    print(x)

# range(start, end, step)
for x in range(1, 20, 5):   # 1, 6, 11, 16
    print(x)

# Dictionary ustidan
car = dict(brand='Ferrari', year=2025)
for key in car:
    print(f'{key}: {car.get(key)}')
```

break va else

break - tsiklni to'xtatadi. else bloki - tsikl BREAK bo'lmay tugasa ishlaydi.

```
for x in range(1, 20, 5):   # 1, 6, 11, 16
    print(x)
    if x > 10:
        print('Break ga yetdi')
        break
else:
    print('Muaffaqiyatli tugatildi')  # break bo'lsa ISHLAMAYDI
```

while tsikli

```
number = 40
while number > 0:
    number -= 10
    print(number)   # 30, 20, 10, 0

# while True - cheksiz tsikl (break bilan to'xtatiladi)
count = 0
while True:
```

```
    count += 1
    x = int(input('Son toping: '))
    if x == 41:
        print(f'Topdingiz! {count} urinishda')
        break
    elif x < 41:
        print(f'{x} dan katta')
    elif x > 41:
        print(f'{x} dan kichik')
```

Eslatma: while True + break - 'game loop' deb ataladi. O'yinlar va interaktiv dasturlarda keng ishlatiladi. continue kalit so'zi - joriy iteratsiyani o'tkazib yuboradi va keyingisiga o'tadi.

continue - iteratsiyani o'tkazib yuborish

```
# Faqat juft sonlarni chiqarish
for i in range(1, 11):
    if i % 2 != 0:
        continue        # toq sonlarni o'tkazib yubor
    print(i)            # 2, 4, 6, 8, 10
```

5. Iterable ob'ektlar va Dictionary

Iterable ob'ektlar

Iterable - ustidan 'for' tsikli bilan aylanish mumkin bo'lgan ob'ektlar. Pythondagi asosiy iterables:

- str - matn ('MIT')
- list - ro'yxat ([1, 2, 3])
- tuple - o'zgarmas ro'yxat ((1, 2, 3))
- dict - lug'at ({key: value})
- range - sonlar oralig'i (range(10))
- map, filter, zip - lazy iteratorlar

```
range_obj = range(3)          # [0, 1, 2]
print(range_obj)              # range(0, 3)

for el in range_obj:
    print(el)                 # 0, 1, 2
```

Dictionary

Dictionary - JSON ob'ektiga o'xshash kalit-qiymat (key-value) tuzilmasi. Java da HashMap, JS da Object ga o'xshaydi.

```
# Literal usul
person = {
    'name': 'Justin',
    'age': 25,
    'single': False
}

# Constructor usul
person_obj = dict(name='Adam', age=20, single=True)

# Qiymat olish
name = person_obj.get('name')          # 'Adam'
hobby = person_obj.get('hobby')        # None (yo'q bo'lsa)
balance = person_obj.get('balance', 0) # 0 (default qiymat)

# O'chirish
del person_obj['single']

# Ustidan aylanish
for key in person_obj:
    print(f'{key}: {person_obj.get(key)}')
```

Dictionary metodlari

```
car = {'brand': 'BMW', 'year': 2024, 'color': 'black'}

# Qo'shish / yangilash
car['speed'] = 250           # yangi kalit qo'shish
```

```
car['year'] = 2025           # mavjudini yangilash

# Barcha key lar
print(car.keys())            # dict_keys(['brand', 'year', ...])

# Barcha value lar
print(car.values())          # dict_values(['BMW', 2025, ...])

# Key-value juftliklari
print(car.items())           # dict_items([('brand', 'BMW'), ...])

# Mavjudligini tekshirish
print('brand' in car)        # True
print('engine' in car)       # False
```

6. Tuples - O'zgarmas ro'yxat

Tuple nima?

Tuple - list ga o'xshash, lekin o'zgarmas (immutable) ma'lumot strukturasi. () qavslar bilan yoki oddiygina vergul bilan yaratiladi.

```
# List - o'zgaradi
fruits = ['apple', 'lemon', 'banana']
fruits[2] = 'melon'         # ishlaydi

# Tuple - o'zgarmaydi!
animals = ('cat', 'dog', 'lion', 'giraffe')
# animals[0] = 'bird'       # TypeError!

tuple_obj = ('MIT', 38, True, 'Adam')   # aralash turlar

# Vergul bilan ham tuple
names = 'Andrew', 'Jack'          # tuple
university = 'Kookmin',           # tuple (bitta element)
```

Unpacking - qiymatlarni ajratish

Unpacking - tuple yoki listdagi qiymatlarni alohida o'zgaruvchilarga birlashtirish.

```
groups = ['MIT', 'FLEXY', 'DEVEX', 'MG']
(x, y, *z) = groups
print(x)    # MIT
print(y)    # FLEXY
print(z)    # ['DEVEX', 'MG']  (* qolganlarini list qilib oladi)
```

*args va **kwargs

*args - o'zgaruvchi miqdordagi argumentlarni tuple sifatida qabul qiladi. **kwargs - nom berilgan argumentlarni dictionary sifatida qabul qiladi.

```
# *args - tuple sifatida keladi
def calculate(*args):
    total = 1
    for x in args:
        total *= x
    print(type(args))    # <class 'tuple'>
    return total

calculate(1, 7, 2, 3)   # 42

# **kwargs - dict sifatida keladi
def introduce(**kwargs):
    print(type(kwargs))  # <class 'dict'>
    print(f"Ismim {kwargs['name']}, yoshim {kwargs['age']}")
```

```
introduce(name='Justin', age=22)
introduce(name='Shawn', age=30, single=True)

# Ikkalasi birga
def greeting(*args, **kwargs):
    print('args:', args)
    print('kwargs:', kwargs)

greeting('hi', True, 10, name='Jon', age=22)
```

zip - ro'yxatlarni birlashtirish

zip() - bir nechta iterableni birlashtirib juftliklar (tuple) hosil qiladi.

```
tuple_1 = (1, 2, 3, 4)
tuple_2 = ('a', 'b', 'c')

zipped = zip(tuple_1, tuple_2)   # lazy iterator
result = list(zipped)
print(result)   # [(1, 'a'), (2, 'b'), (3, 'c')]

# zip qisqa ro'yxat bilan cheklanadi (3 ta, chunki tuple_2 = 3 element)
```

Eslatma: zip() lazy iterator - faqat list(), tuple() yoki for bilan ishlatilganda ishga tushadi. Ikki ro'yxatni parallel aylanishda juda qulay.

7. List - Ro'yxat

List yaratish

List - tartibli, o'zgaruvchan (mutable), takrorlanadigan ma'lumot strukturasi.

```
# Literal
groups = ['MIT', 'FLEXY', 'DEVEX', 'MG']

# Constructor
result = list('Welcome!')   # ['W', 'e', 'l', 'c', 'o', 'm', 'e', '!']

# Indexing va Slicing
fruits = ['apple', 'melon', 'strawberry', 'orange']
print(fruits[0])       # apple (birinchi element)
print(fruits[-1])      # orange (oxirgi element)
print(fruits[0:2])     # ['apple', 'melon'] (0 dan 2 gacha, 2 kiritilmaydi)
print(fruits[::2])     # ['apple', 'strawberry'] (har 2-element)
print(fruits[::-1])    # ['orange', 'strawberry', 'melon', 'apple'] (teskari)
```

List metodlari (mutable)

```
letters = ['a', 'd', 'e']

letters.append('c')        # oxiriga qo'shadi
letters.insert(0, 'z')     # indeks bo'yicha qo'shadi
letters.pop()              # oxirini olib chiqaradi
letters.pop(0)             # boshini olib chiqaradi
letters.remove('d')        # qiymat bo'yicha o'chiradi
letters.clear()            # barchasini tozalaydi

# del - kesib tashlash
animals = ['dog', 'cat', 'zebra', 'fish', 'lion', 'snake']
del animals[2:4]           # 2 va 3 indexdagilarni o'chiradi

# Mavjudligini tekshirish
if 'cat' in animals:
    print(animals.index('cat'))   # indeksini topadi

# Saralash
numbers = [2, 32, 20, 4, 7]
numbers.sort()             # joyida saralaydi (mutable)
numbers.sort(reverse=True) # teskari tartibda

# sorted() - yangi ro'yxat qaytaradi (immutable)
new_nums = sorted(numbers) # original o'zgarmaydi
```

Lambda funksiya

Lambda - kichik, nomi yo'q (anonim) funksiya. sort(), map(), filter() bilan birgalikda juda qulay.

```
# Oddiy funksiya
def calc(x, y): return x * y
print(calc(3, 5))   # 15

# Lambda bilan ekvivalent
calc2 = lambda x, y: x * y
print(calc2(3, 5))  # 15

# Sort with lambda
people = [('Justin', 26), ('Martin', 35), ('Adam', 20)]
people.sort()                          # alfabetik (ismi bo'yicha)
people.sort(key=lambda p: p[1])        # yosh bo'yicha
people.sort(key=lambda p: p[1], reverse=True)  # teskari
```

enumerate, map va filter

```
# enumerate() - index va qiymatni birga beradi
animals = ['dog', 'cat']
for el in enumerate(animals):
    print(el)             # (0, 'dog'), (1, 'cat')

for (index, value) in enumerate(animals):
    print(f'{index}: {value}')

# dict.items() - dict uchun xuddi shunday
car = dict(brand='Ferrari', year=2026)
for key, value in car.items():
    print(f'{key}: {value}')

# map() - har elementga funksiya qo'llaydi
cars = [('Ferrari', 78), ('Toyota', 87), ('Audi', 116)]
names = list(map(lambda car: car[0], cars))
print(names)    # ['Ferrari', 'Toyota', 'Audi']

# filter() - shartga mos elementlarni tanlaydi
fast_cars = list(filter(lambda car: car[1] > 80, cars))
print(fast_cars)  # [('Toyota', 87), ('Audi', 116)]
```

Maslahat: map() va filter() lazy iterator qaytaradi - list() bilan o'rash kerak. List comprehension ham xuddi shu ishni qiladi: [car[0] for car in cars]

List Comprehension (qo'shimcha)

```
# Oddiy
squares = [x**2 for x in range(1, 6)]
print(squares)   # [1, 4, 9, 16, 25]

# Shartli
evens = [x for x in range(10) if x % 2 == 0]
```

```
print(evens)     # [0, 2, 4, 6, 8]

# map() ning ekvivalenti
cars = [('Ferrari', 78), ('Toyota', 87)]
names = [car[0] for car in cars]
print(names)     # ['Ferrari', 'Toyota']
```

8. Funksiyalar

Define va Call

Funksiya - qayta ishlatiladigan kod bloki. def kalit so'zi bilan aniqlanadi (define), keyin chaqiriladi (call). Pythonda blok {} emas, indentation (:) bilan belgilanadi.

```
# DEFINE - qurilish
def greet(a):           # void - return yo'q, None qaytaradi
    print(f'Salom, {a}')

def greeting(b):        # return bor
    print('greeting bajarildi')
    return f'Hi {b}'

# CALL - chaqirish
result1 = greet('Adam')       # None qaytaradi
print('result1:', result1)    # None

result2 = greeting('Justin')
print('result2:', result2)    # Hi Justin
```

Parametr va Argument turlari

```
# Default argument
def give_greet(name, age=20):       # age=20 default qiymat
    return f'Salom {name}, yoshingiz {age}'

# Keyword argument
print(give_greet(name='AJAX', age=22))
print(give_greet('Bob'))            # age=20 default ishlatiladi

# Positional argument
print(give_greet('Tom', 30))        # tartib muhim
```

Scope - o'zgaruvchilar doirasi

Scope - o'zgaruvchi ko'rinadigan hudud. Python LEGB qoidasini ishlatadi: Local > Enclosing > Global > Built-in.

```
b = 100   # Global scope

def calculate(a, b=1):   # b=1 parametri global b ni yopadi (local)
    c = a * b             # c - local scope
    print(f'c = {c}')

calculate(5)     # c = 5  (b=1 default)
calculate(5, 3)  # c = 15 (b=3)
# print(c)       # NameError - c tashqarida ko'rinmaydi

# global kalit so'zi - funksiya ichidan global o'zgaruvchini o'zgartirish
```

```
count = 0
def increment():
    global count
    count += 1

increment()
print(count)     # 1
```

Funksiyalar haqida qo'shimcha

```
# Funksiya ham ob'ekt - boshqa funksiyaga argument sifatida beriladi
def apply(func, value):
    return func(value)

result = apply(str.upper, 'hello')
print(result)    # HELLO

# Nested funksiya
def outer(x):
    def inner(y):
        return x + y
    return inner

add5 = outer(5)
print(add5(3))   # 8
```

9. Classlar - Asosiy

Class nima?

Class - ob'ekt yaratish uchun shablon (blueprint). Har bir class uchta asosiy qismdan iborat: state (holat), constructor, method.

```
class Person:
    # state (class darajasidagi holat)
    message = 'class state property'

    # constructor - ob'ekt yaratilganda ishlaydi
    def __init__(self, name, age):
        self.name = name    # instance state
        self.age = age

    # method
    def introduce(self):
        print(f'{self.name} says: Salom!')

    def say_age(self):
        print(f'{self.name} - {self.age} yoshda')

    @classmethod
    def explain(cls):
        print('Static method ishladi')

# Ob'ektlar yaratish
person1 = Person('Justin', 25)
person2 = Person('Martin', 35)

person1.introduce()      # Justin says: Salom!
person2.say_age()        # Martin - 35 yoshda

# Class (static) property
print(Person.message)    # class state property
Person.explain()         # Static method ishladi
```

Magic / Dunder metodlar

Magic metodlar __ (double underscore - dunder) bilan boshlanadi va tugaydi. Python ularni maxsus holatlarda avtomatik chaqiradi.

```
class Car:
    description = 'This class makes cars'

    def __new__(cls, *args):     # ob'ekt yaratilishdan oldin
        print('__new__ ishladi')
        return super().__new__(cls)
```

```
    def __init__(self, name, year):
        self.name = name
        self.year = year

    def __str__(self):           # print() chaqirilganda
        return f'{self.name} - {self.year} yil'

    def __call__(self):          # ob'ekt funksiya sifatida chaqirilganda
        print('Ob'ekt funksiya sifatida chaqirildi!')
        return True

my_car = Car('BMW', 2024)
print(my_car)           # BMW - 2024 yil  (__str__ ishlaydi)
result = my_car()       # __call__ ishlaydi
print(result)           # True
```

Asosiy magic metodlar

- __init__ - ob'ekt yaratilganda (constructor)
- __new__ - ob'ekt xotiraga olishdan oldin
- __str__ - print() chaqirilganda, str() qilinganda
- __call__ - ob'ekt () bilan chaqirilganda
- __len__ - len() chaqirilganda
- __eq__ - == operatori ishlatilganda
- __getitem__ - [] indeks bilan kirganda

10. OOP - Encapsulation, Inheritance, Polymorphism

Encapsulation - Ma'lumotni yashirish

Encapsulation - ma'lumotni tashqaridan himoya qilish. Python da: public (oddiy), __private (ikki pastki chiziq), _protected (bir pastki chiziq).

```
class Account:
    description = 'Bank account class'

    def __init__(self, owner, amount):
        self.__owner = owner    # __private (tashqaridan ko'rinmaydi)
        self.__amount = amount

    def get_balance(self):
        print(f'{self.__owner} da {self.__amount} USD')

    def deposit(self, amount):
        self.__amount += amount

    def withdraw(self, amount):
        self.__amount -= amount

my_account = Account('Shawn', 1000)
my_account.get_balance()      # Shawn da 1000 USD
my_account.deposit(5300)
my_account.withdraw(800)
my_account.get_balance()      # Shawn da 5500 USD

# Private ga to'g'ridan-to'g'ri kirish - xato!
try:
    result = my_account.__amount
except AttributeError:
    print('Private ga kirish mumkin emas')
```

Inheritance - Meros

Inheritance - Parent (ota) classdan Child (bola) class yaratish. Bola class ota classning barcha public va protected property/methodlarini oladi.

```
class Animal:  # PARENT
    def __init__(self, voice):
        self.__status = 'Tirik'
        self.voice = voice

    def make_voice(self):
        print(f'Ovoz: {self.voice}')

class Dog(Animal):  # CHILD
```

```
    def __init__(self, name, sound, voice):
        self.name = name
        self.sound = sound
        super().__init__(voice)  # ota classni ishga tushiradi

    def introduce(self):
        print(f'{self.name}: {self.sound}-{self.sound}')

    def make_voice(self):        # Override - ota methodini qayta yozadi
        print(f'{self.name} deydi: {self.sound}')

dog = Dog('Rex', 'Woof', True)
dog.introduce()    # Rex: Woof-Woof
dog.make_voice()   # Rex deydi: Woof
```

isinstance va issubclass

```
fish = Fish('Nemo', 'ZzZ', False)

# isinstance - ob'ekt berilgan classga tegishlimi?
print(isinstance(fish, Fish))    # True
print(isinstance(fish, Animal))  # True (merosiy)
print(isinstance(fish, object))  # True (hamma object)

# issubclass - class boshqa classning farzandimi?
print(issubclass(Fish, Animal))  # True
print(issubclass(Animal, object))# True
```

Polymorphism - Ko'p shakllilik

Polymorphism - bir xil method nomi turli classlarda turlicha ishlashi. Inheritance orqali method override qilinadi.

```
class Cat(Animal):
    def __init__(self, name, sound, voice):
        self.name = name
        self.sound = sound
        super().__init__(voice)

    def make_voice(self):
        print(f'{self.name} miyov deydi: {self.sound}')

class Fish(Animal):
    def make_voice(self):
        print(f'{self.name} - jim (baliq!)')

# Polymorphism - bir xil method, turlicha xatti-harakat
animals = [
    Dog('Rex', 'Woof', True),
    Cat('Tom', 'Meow', True),
    Fish('Nemo', 'ZzZ', False)
```

```
]
for animal in animals:
    animal.make_voice()   # har biri o'ziga xos ishlaydi
```

Maslahat: OOP 4 pillar: Abstraction (murakkablikni yashirish), Encapsulation (ma'lumotni himoyalash), Inheritance (meros olish), Polymorphism (ko'p shakllilik).

11. Xatolarni Boshqarish - Error Handling

try / except / else / finally

Xatolarni boshqarish dasturning to'satdan to'xtab qolishini oldini oladi. try - bajarilishi mumkin bo'lgan kod, except - xato bo'lganda, else - xato bo'lmaganda, finally - har doim bajariladi.

```
car_dict = dict(name='Toyota', year=2026, electric=True)

try:
    a = car_dict.speed           # AttributeError
    result = car_dict['origin']  # KeyError
    print('result:', result)
except (KeyError, AttributeError) as err:
    print('Xato:', err)
else:
    print('Xatosiz bajarildi')
finally:
    print('Har doim bajariladi (resurslarni yopish)')
```

Asosiy xato turlari

- TypeError - noto'g'ri tur ('2' + 2)
- ValueError - noto'g'ri qiymat (int('abc'))
- KeyError - dict da yo'q kalit (d['notkey'])
- IndexError - ro'yxatda yo'q indeks ([1,2][5])
- AttributeError - ob'ektda yo'q metod/holat
- NameError - aniqlanmagan o'zgaruvchi
- ZeroDivisionError - nolga bo'lish

```
# Maxsus xato ko'tarish
def divide(a, b):
    if b == 0:
        raise ValueError('Nolga bo\'lish mumkin emas!')
    return a / b

try:
    result = divide(10, 0)
except ValueError as e:
    print(e)   # Nolga bo'lish mumkin emas!
```

Tezkor Ma'lumotnoma - Quick Reference

Ma'lumot turlari

```
int    -> 100, -5, 0
float  -> 3.14, -2.7
str    -> 'text', "text"
bool   -> True, False
None   -> None
list   -> [1, 'a', True]
tuple  -> (1, 'a', True)
dict   -> {'key': 'value'}
set    -> {1, 2, 3}
```

Operatorlar

```
Arifmetik:   +  -  *  /  //  %  **
Taqqoslash:  ==  !=  >  <  >=  <=
Mantiqiy:    and  or  not
Identity:    is  is not
Membership:  in  not in
O'zlashtirish: +=  -=  *=  /=  //=  %=  **=
```

String metodlari

```
s = 'Hello World'
s.upper()        -> 'HELLO WORLD'
s.lower()        -> 'hello world'
s.title()        -> 'Hello World'
s.strip()        -> bo'sh joylarsiz
s.split(' ')     -> ['Hello', 'World']
' '.join(['a','b']) -> 'a b'
s.replace('o','0')  -> 'Hell0 W0rld'
s.startswith('He')  -> True
s.endswith('ld')    -> True
len(s)              -> 11
```

List metodlari

```
lst = [3, 1, 4, 1, 5]
lst.append(9)     # [3,1,4,1,5,9]
lst.insert(0, 0)  # [0,3,1,4,1,5,9]
lst.pop()         # oxirini chiqaradi
lst.pop(0)        # boshini chiqaradi
lst.remove(1)     # birinchi 1 ni o'chiradi
lst.sort()        # joyida saralaydi
lst.reverse()     # joyida teskari
lst.index(4)      # 4 ning indeksi
lst.count(1)      # 1 nechta bor
sorted(lst)       # yangi saralangan list
```

```
len(lst)          # elementlar soni
```

Dictionary metodlari

```
d = {'a': 1, 'b': 2}
d.get('a')           # 1
d.get('c', 0)        # 0 (default)
d.keys()             # dict_keys(['a','b'])
d.values()           # dict_values([1, 2])
d.items()            # dict_items([('a',1),('b',2)])
d.update({'c': 3})   # qo'shish/yangilash
d.pop('a')           # 'a' ni o'chiradi, qiymatini qaytaradi
'a' in d             # True
```